

Hybrid Documents RAG-based Chatbot

Course: CSE299 Section:01 Group 3

Maharun Afroz, 2211023642, Sanjida Amin Nadia, 2122319642,
Tasnia Hossain, 2121480642, and Tahmina Mozumdar, 2111482642

Abstract—The swift expansion of digital documents has created a growing demand for intelligent systems that interactively retrieve pertinent information from various sources. This project introduces a hybrid chatbot based on Retrieval-Augmented Generation (RAG) that combines conventional search methods with a generative language model to deliver conversational responses tailored to user inquiries. The chatbot handles document collections, ranks relevant materials utilizing vector embeddings, and produces contextually relevant answers. Built with an easy-to-use Streamlit interface, a FastAPI backend, and MongoDB database for storing conversations. The system showcases effective hybrid search capabilities, real-time engagement, and offline functionality without depending on cloud services. Experimental findings reveal that the system efficiently sources precise information from multi-source documents, presenting notable implications for educational, research, and enterprise applications.

Chapter 1 Introduction

1.1 Background and Motivation

In an era characterized by an explosion of information, individuals frequently find themselves inundated with the sheer volume of data dispersed across various documents and formats. Conventional search engines fall short in their capacity to grasp context and deliver coherent, conversational responses derived from cross-document analysis. Recent progress in artificial intelligence, especially in Retrieval-Augmented Generation (RAG), has opened the door for systems that merge document retrieval with natural language generation to offer intelligent, context-sensitive replies. Nonetheless, many current implementations either depend on cloud-based solutions or are tailored for single-source document retrieval, lacking the adaptability and efficiency required in offline, hybrid settings.

To fill this void, we propose a RAG-based chatbot that facilitates interactive, context-aware information retrieval from diverse document sources. This system is particularly advantageous for environments with restricted internet connectivity, such as educational institutions, research laboratories, or corporate knowledge management systems. Our inspiration arises from the necessity to develop an offline-capable, efficient, and user-friendly system that integrates search and generative AI to enhance decision-making and learning.

1.2 Purpose and Goal of the Project

The objective of this project is to design and create a hybrid document RAG-based chatbot that:

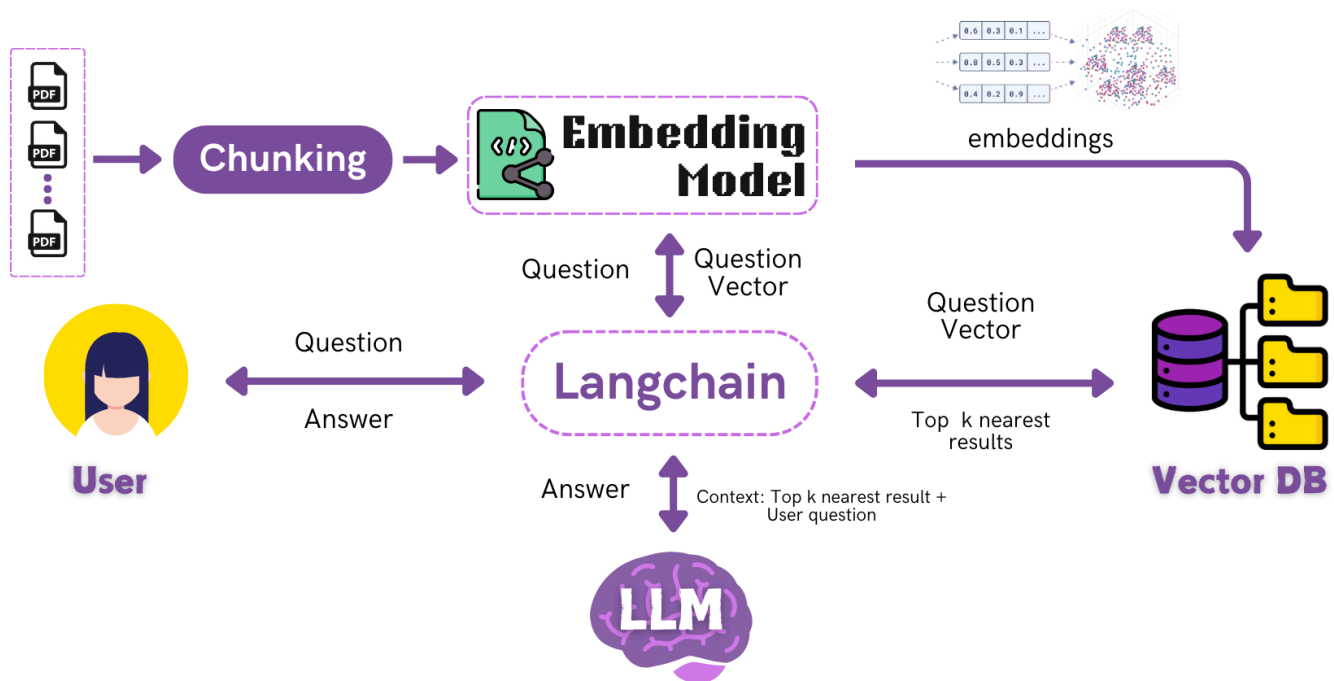
- Accepts user inquiries and extracts pertinent content from multiple chosen PDFs or documents.
- Utilizes a ranking system to identify the most relevant excerpts.
- Implements a generative model to deliver coherent responses within a conversational framework.
- Operates offline without reliance on cloud services.
- Offers both frontend and backend integration with a modular design.
- Offers storage to save, load, and delete previous chat sessions.

The innovation of this project lies in its ability to handle hybrid documents, function offline, and seamlessly integrate vector-based document retrieval with response generation. In contrast to standard RAG implementations, our system allows for selective document targeting, session-based interaction, and re-ranking of outputs before generation, making it well-suited for practical, real-world applications.

1.3 Organization of the Report

This report is organized into the following chapters:

Chapter 1: Introduces the background, motivation, problem statement, objectives, and overall significance of the Hybrid Documents RAG-Based Chatbot project.



Chapter 2: Reviews the current research landscape of Retrieval-Augmented Generation (RAG)-based systems and chatbot implementations. It explores existing advancements and outlines the challenges faced in processing large-scale and unstructured data.

Chapter 3: Details the methodology used in the project, including system design, architecture, development tools, and implementation strategies to build the proposed chatbot.

Chapter 4: Presents the experimental results, system performance analysis, and output behavior of the chatbot when interacting with various sample queries.

Chapter 5: Examines the project's impact on societal, health, safety, legal, and cultural aspects. It also discusses the environmental and sustainability implications of the chatbot system.

Chapter 6: Describes the project timeline and resource planning. A Gantt chart illustrates the task distribution from January to April, along with a breakdown of potential costs and resource requirements.

Chapter 7: Addresses Complex Engineering Problems (CEP) and Complex Engineering Activities (CEA), highlighting the depth of engineering knowledge, design complexity, and real-world considerations involved in the project.

Chapter 8: Concludes the report with a summary of the work completed, outlines key limitations and provides

recommendations for future improvements, such as multilingual support and enhanced reasoning capabilities.

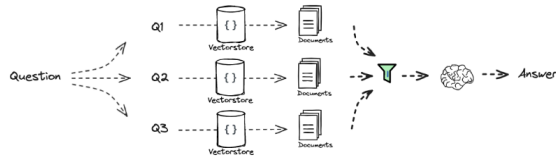
Chapter 2 Review of Research Literature

2.1 Existing Research and Limitations

The demand for innovative, context-sensitive chatbots has spurred document retrieval and natural language generation investigations. Conventional document search engines, such as Elasticsearch or models based on TF-IDF, utilize keyword-based searching, which frequently produces irrelevant or incomplete answers due to insufficient contextual comprehension. To enhance this, contemporary retrieval-augmented systems have become increasingly popular.

A notable method is the RAG (Retrieval-Augmented Generation) model, as introduced by Lewis et al. [1]. This framework integrates a dense retriever—commonly employing FAISS or DPR (Dense Passage Retrieval)—with a generative model like BART T5 or other LLM models, enabling the system to fetch pertinent documents and generate responses in natural language. Their model surpassed traditional question-answering models on various benchmarks, including Natural Questions and WebQuestions.

Karpukhin et al. [2] improved dense retrieval through a dual encoder framework, training questions, and passage embeddings independently to enhance semantic alignment. Other research has examined hybrid search strategies, merging sparse and dense retrieval (e.g., ColBERT, Anserini+T5) to balance precision and recall.



Nevertheless, these systems typically necessitate robust cloud infrastructure and are generally trained on extensive public datasets rather than tailored user documents.

Numerous open-source initiatives, such as Haystack by Deepset and LangChain by Harrison Chase, have created pipelines for RAG-based applications. These frameworks facilitate custom document ingestion, embedding, and conversational agents, yet they are often optimized for cloud environments and tend to lack immediate support for hybrid offline systems.

Recent research, such as that by Wang et al. [3], illustrates that context-aware QA systems significantly enhance user experience when implemented in educational and legal settings. However, their systems presume uninterrupted Internet access and centralized document collections.

The following shortcomings have been noted in the existing literature:

- (1) Numerous systems rely on cloud services like OpenAI APIs, rendering them impractical for offline use.
- (2) Most studies concentrate on public knowledge bases and fail to manage hybrid or user-uploaded documents effectively.
- (3) Few implementations offer adaptability in choosing specific documents for each session or incorporating multi-format document ingestion.
- (4) Re-ranking strategies to guarantee high-relevance content are often neglected in favor of straightforward top-k retrieval.

These shortcomings highlight the primary motivation behind our proposed system. By creating a self-contained hybrid document chatbot, our project broadens the RAG concept to encompass an offline, session-based, and multi-document environment with selective retrieval and re-ranking strategies. The chatbot enables users to engage with uploaded documents—such as PDFs, manuals, or notes—and receive accurate responses without dependence on the cloud.

Chapter 3 Methodology

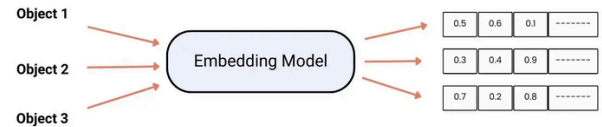
3.1 System Design

The design of the Hybrid Documents RAG-Based Chatbot employs a modular framework comprising four key components: document preprocessing, vector store retrieval, re-ranking, and natural language generation. This complete system is crafted to function effectively in both online and offline environments, ensuring high precision and context retention.

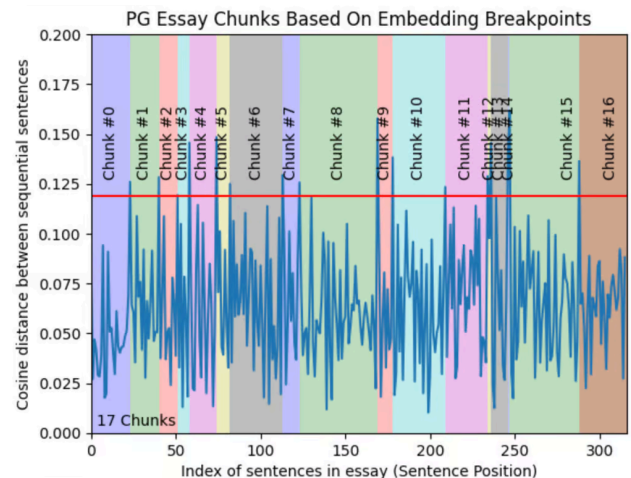
Figure 1 depicts the block diagram of the overall system architecture.

System Flow Overview:

1. **Document Ingestion:** Users can upload multiple documents in PDF format.



2. **Chunking & Embedding:** The text is extracted and segmented into logical pieces. Each segment is transformed into a vector format using a sentence transformer (nomic-embed-text).



- 3. **Vector Retrieval:** A local vector database (FAISS - Facebook AI Similarity Search) is accessed to obtain the top-k relevant segments.
- 4. **Re-ranking:** The acquired segments are ordered based on various models (default, ms-macro-MiniLM-L-12-v2, rank-t5-flan, ms-macro-MultiBERT-L-12, etc.).
- 5. **Response Generation:** The highest-ranked segments and the user’s inquiry are sent to a language model (like deepseek-r1, llama3.1, gemma3, qwen2.5, etc) to produce a coherent response.

3.2 Hardware and/or Software Components

The project uses minimal hardware but includes a stack of key software tools for backend, frontend, and document handling. The table below summarizes the significant tools used.

Table 1: Software/Hardware Tools Used

Tool	Functions	Other similar Tools (if any)	Why Selected
FastAPI	Backend API	Flask, Django	Lightweight, fast, and easy to integrate with modern Python libraries
Streamlit	Frontend Interface	React, Dash	Quick UI building for ML/AI applications
llama 3.2, deepseek-r1, gemma3 etc	Language Model Integration	Cohere, Claude, LLaMA	High-quality text generation
nomic-embed-text	Vector Embeddings	OpenAI embeddings, GloVe	State-of-the-art sentence-level semantic embeddings
FAISS	Vector Database	ChromaDB	Efficient nearest neighbor search for dense vectors
PyPDFLoad	PDF	textract,	Accurate and

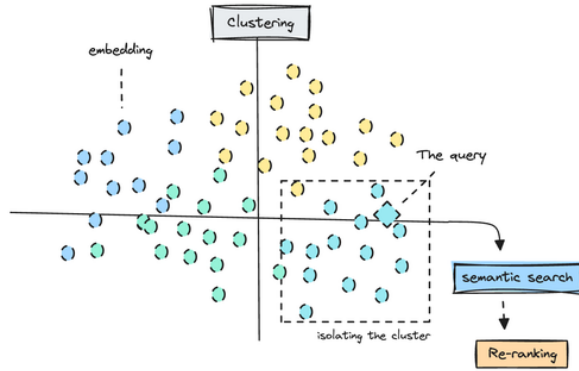
er	Processing	tika	fast PDF text extraction
MongoDB	Store sessions and conversation , also stores Vector Store	Firebase, PostgreSQL	NoSQL database for session storage and user management
FlashRank	Reranker	ML-based re-rankers	Reranks retrieved documents based on semantic closeness
Ollama	Local Language model providers.	Llama index, LM Studio etc.	Ollama is one of the most used LLM providers.

3.3 Hardware and/or Software Implementation

The system was developed in a modular fashion to enhance testing and scalability. Key implementation aspects include:

- The frontend was created using Streamlit, enabling users to upload documents, review conversation history, and pose questions.
- The backend was built with FastAPI, which provides endpoints for document ingestion, embedding generation, session management, and question answering.
- Embedding Generation utilizes SentenceTransformer (all-MiniLM-L6-v2) to produce 384-dimensional dense vectors for each segment.
- The Chunking Strategy divides documents into overlapping segments (e.g., 200–300 words) to improve context retention.
- Vector Search is managed locally with FAISS, ensuring the system can operate without internet access.
- Answer Generation leverages OpenAI’s GPT-3.5 API in online mode. For offline use, the system supports open-source models such as LLaMA or Mistral.
- Session Control lets users interact with specific documents and previous inquiries via MongoDB.

- **ROUGE-2:** 0.2317
- **ROUGE-L:** 0.4007



Chapter 4 Investigation/Experiment, Result, Analysis, and Discussion

4.1 Experimental Design

We created a total of **488** questions, answer datasets along with response time, question source, etc. We used this dataset for both quantitative and qualitative methods. While the quantitative method works word-by-word, the qualitative method (like RAGAS[7]) focuses on underlying meaning.

4.2 Standard Qualification Metrics

The system was evaluated using standard classification metrics. Below are the formulas used:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Results:

- **Accuracy:** 0.0336
- **Precision:** 0.0336
- **Recall:** 1.0000
- **F1 Score:** 0.0651
- **Hit Rate:** 0.1469

Language Evaluation Scores:

- **BLEU:** 0.0715
- **ROUGE-1:** 0.4319

4.3 Qualitative Evaluation (DeepEval)

The chatbot's answer quality was also evaluated using the deepeval framework, which checks for faithfulness, contextual understanding, and relevance.

Metric	Score
Faithfulness	0.672
Answer Relevancy	0.928
Contextual Precision	1.000
Contextual Recall	1.000
Contextual Relevancy	0.398

The results show high contextual accuracy and recall, with good relevance and moderate faithfulness. These metrics indicate the chatbot provides highly relevant and coherent answers in most cases, with room for improvement in output diversity and abstraction.

DeepEval metrics works better as it focuses on the underlying meaning rather than literal words. It uses the RAGAS[7] method to evaluate tests.

Chapter 5 Impacts of the Project

5.1 Impact of this Project on Societal, Health, Safety, Legal, and Cultural Issues

The RAG-based hybrid document chatbot significantly contributes to various societal dimensions, particularly information accessibility, data privacy, and digital inclusion. By enabling users to engage with multiple documents concurrently, even offline, this system fosters equitable access to information in contexts where internet connectivity is limited or unreliable. This feature benefits rural or underserved regions, educational institutions, and sensitive legal or corporate settings.

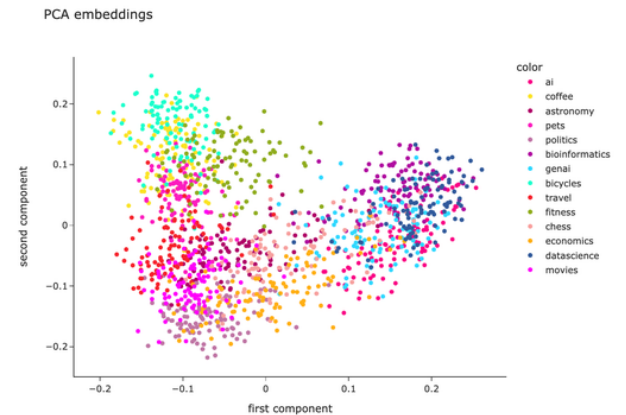
From a health and safety standpoint, the offline functionality of the chatbot facilitates the secure management of sensitive data, such as medical or legal records, thereby mitigating the risk of unauthorized data breaches and bolstering user trust. When confidential documents such as patient reports, psychological evaluations, or legal evidence must be examined, the system reduces dependency on cloud-based services that are susceptible to external threats. This initiative addresses data protection and privacy concerns by endorsing local

data processing. In adherence to privacy regulations, such as the General Data Protection Regulation (GDPR), the system empowers users to

manage and analyze their data independently without necessitating transmission to third-party servers. This design choice grants users enhanced control over their personal or organizational information, contributing to ethical technology utilization.

Culturally, the chatbot promotes inclusivity by accommodating hybrid documents encompassing various formats and languages. This feature is especially beneficial in multilingual regions or culturally diverse workplaces, fostering understanding and collaboration. Moreover, the capacity to engage in context-aware, document-specific dialogues serves educational purposes across diverse cultural landscapes, promoting learning and knowledge-sharing without digital constraints.

In conclusion, this project not only improves the efficiency of document interactions but also aligns with broader social objectives, including digital equity, privacy-centric design, and the adoption of inclusive technology.



5.2 Impact of this Project on Environment and Sustainability

While primarily a software-based system, the RAG-based hybrid document chatbot significantly advances environmental sustainability by efficiently utilizing digital resources and reducing reliance on cloud infrastructure. By facilitating offline operation, the system effectively reduces the need for continuous server communication and cloud computing, which are often linked to high energy consumption and carbon emissions from large-scale data centers.

Moreover, the project fosters decreased paper usage by promoting local document processing. In numerous educational, legal, and corporate settings, users have traditionally depended on printed documents for review

and reference. This chatbot encourages a transition toward paperless workflows by providing an interactive digital alternative, aiding resource conservation and minimizing paper waste.

The project's support for hybrid documents further ensures excellent document compatibility, extending the usable life of existing digital files and mitigating the need for repetitive data reformatting or duplication. This contributes to a reduction in the overall digital footprint by preventing unnecessary reprocessing or storage replication.

The system's design also incorporates lightweight components appropriate for local deployment, rendering it accessible on low-power devices. This facilitates energy-efficient usage, particularly in regions with limited computational resources or where sustainability is a primary concern.

Chapter 6 Project Planning and Budget

Here is a Gantt chart illustrating our project's timeline and task distribution across the months. It visually represents the planning, development, and delivery phases for better project tracking.

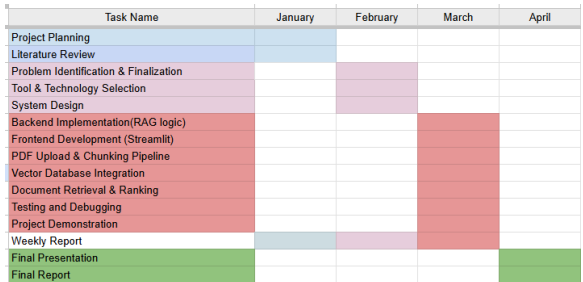


Figure: A Grantt Chart of the project

Budget

This project was developed entirely using free and open-source resources, including Python libraries, the Streamlit framework for the frontend, and locally hosted LLMs via Ollama. No additional hardware or paid services were required. The only notable cost is electricity usage during development and testing. Assuming a 50W power

consumption, running the system continuously for 24 hours would result in an estimated cost of 0.156 USD per day. Overall, the project is highly cost-effective and accessible for further development or deployment.

Chapter 7 Complex Engineering Problems and Activities

7.1 Complex Engineering Problems (CEP)

The development of the Hybrid Documents RAG-Based Chatbot involves several Complex Engineering Problems due to the integration of advanced technologies like document chunking, vector databases, offline LLMs, and session-based interactions. It presents conflicting requirements, such as maintaining accuracy while supporting offline use and data privacy. The project requires expertise in NLP, embeddings, vector search, and prompt engineering, as well as the integration of frontend and backend components. It is multidisciplinary, involving AI, software engineering, and cybersecurity, while also addressing legal and ethical concerns. With no single correct solution, the project clearly reflects the attributes of Complex Engineering Problems.

TABLE II. COMPLEX ENGINEERING PROBLEM ATTRIBUTES TABLE

Attributes		Addressing the Complex Engineering Problems (P) in the Project
P1	Depth of knowledge required (K3-K8)	The project requires knowledge of Python programming, NLP, frontend/backend development, information retrieval, document processing (PDF parsing), embedding techniques, and user session management.
P2	Range of conflicting requirements	The system must balance performance (fast response time) and accuracy (relevant document matching), while also maintaining offline capability without compromising memory usage.

Attributes		Addressing the Complex Engineering Problems (P) in the Project
P1	Depth of knowledge required (K3-K8)	The project requires knowledge of Python programming, NLP, frontend/backend development, information retrieval, document processing (PDF parsing), embedding techniques, and user session management.
P3	Depth of analysis is required	Selecting the right vector database, embedding models, and chunking strategy requires deep analysis to optimize retrieval relevance and processing speed.

7.2 Complex Engineering Activities (CEA)

The tasks related to creating and executing this chatbot also fulfill the requirements for intricate engineering efforts. These encompass a variety of resources, creative ideas, and social/environmental significance.

TABLE III. COMPLEX ENGINEERING PROBLEM ACTIVITIES TABLE

Attributes		Addressing the Complex Engineering Activities (A) in the Project
A1	Range of resources	The project involves human resources (developers), software (Python, Streamlit/React, FastAPI), hardware (PCs with local hosting), and modern tools (PDF parsers, vector DBs, embeddings).
A4	Consequences to society/environment	The tool supports learning and research by enabling quick and accurate document-based information retrieval, promoting digital literacy and productivity.

Chapter 8 Conclusions

8.1 Summary

This project aimed to create a Hybrid Documents Retrieval-Augmented Generation (RAG)--based Chatbot that facilitates conversational interactions with a knowledge base derived from multiple uploaded documents. This system employs a Retrieval-Augmented Generation approach to deliver pertinent responses using semantic document retrieval and generative text modeling techniques. Essential components of the system include document chunking, embedding generation, vector database storage, and an intuitive frontend for user query interaction. The chatbot functions offline, ensuring data privacy and autonomy from cloud infrastructure while offering precise and contextually relevant responses based on the content retrieved from the documents.

8.2 Limitations

Notwithstanding the successful implementation of the project, several limitations remain evident. Firstly, the quality of responses is significantly influenced by the quality and organization of the uploaded documents. Additionally, the system may encounter difficulties addressing queries requiring intricate reasoning across multiple documents. Furthermore, response times may escalate as the volume of documents increases, mainly when operating on constrained local hardware. The current version is also noteworthy that advanced features such as document summarization, voice interface, or multilingual support are absent. Lastly, the existing document chunking and re-ranking strategies may not effectively accommodate long or unstructured texts.

8.3 Future Improvements

To further enhance the system, the following improvements are proposed:

1. Multilingual Support: Implement language detection and translation capabilities to ensure broader accessibility for users.
2. Enhanced Retrieval: Employ more sophisticated re-ranking models or fine-tuned embeddings to improve the relevance of search results.
3. Summarization Module: Introduce an automated summarization feature for documents or retrieved content, facilitating quick reference.
4. Voice Interface: Integrate speech-to-text and text-to-speech technologies to allow for hands-free interaction.

5. Cross-Document Reasoning: Develop logic that enables drawing inferences from multiple sources, enhancing the system's ability to address complex queries.

6. Scalability: Optimize the backend infrastructure to efficiently manage larger datasets using memory mapping or distributed processing techniques.

Reference

- [1] "Evaluation of Retrieval-Augmented Generation: A Survey," arXiv, 3 Jul 2024. Available: <https://arxiv.org/abs/2405.07437>.
- [2] "Evaluating Retrieval Quality in Retrieval-Augmented Generation," ACM, 11 Jul 2024. Available: <https://dl.acm.org/doi/abs/10.1145/3626772.3657957>.
- [3] Y. Xi, W. Liu, X. Dai, R. Tang, W. Zhang, Q. Liu, X. He, and Y. Yu, "Context-aware reranking with utility maximization for recommendation," arXiv preprint arXiv:2110.09059, Feb. 2022.
- [4] OpenAI, "ChatGPT," chat.openai.com. [Online]. Available: <https://chat.openai.com/>
- [5] Google, "Gemini," gemini.google.com. [Online]. Available: <https://gemini.google.com/>
- [6] "How to Create a Gantt Chart in Google Sheets", YouTube, Feb.11, 2019.[Online]. Available: https://youtu.be/8eKkoM2zGIk?si=nHnK2_q2VUAIGdwD
- [7] "RAGAs: Automated Evaluation of Retrieval Augmented Generation" (<https://aclanthology.org/2024.eacl-demo.16/>) (Es et al., EACL 2024)
- [8] LangChain (<https://python.langchain.com/docs/>)
- [9] DeepEval (<https://www.deepeval.com/docs/>)